



B.U.T. Informatique

Parcours A
Année 2023-2024

Rapport d'alternance : Développement d'un logiciel de supervision énergétique multisites

Étudiant : Alexandre Maurice

Alternance réalisée chez **InvisoO**



Date :
Du 04 sept. 2023 au 02 sept. 2024

Encadrement :
M. Vincent Agi

Enseignant référent : M. Philippe Roose

Lecteur : M^{me} Margaret Borthwick

Année universitaire 2023-2024 (Semestre 5-6)

Abstract. This report presents the professional experience gained during my apprenticeship at Invisio, an innovative company founded in 2023. Invisio is dedicated to developing a multisite energy monitoring solution designed to meet the requirements of the BACS Decree (Building Automation and Control Systems) and the Tertiary Decree. These decrees aim respectively to improve the energy efficiency of buildings through the automation and control of systems, and to reduce energy consumption in the tertiary sector.

Within a small dynamic team, I participated in the complete development of a software solution, from its initial design to its implementation. Our main goal was to provide local entities, such as community councils, municipalities, and local businesses, with a reliable and affordable energy monitoring tool. We adopted an incremental development approach, enabling simplified and rapid deployment, achievable almost with a single click. This methodology allowed us to continuously test and improve our product, ensuring a better response to the needs of our end users.

Optimizing the architecture and data storage was a central focus of our development. By optimizing the infrastructure, we were able to ensure the scalability of our solution while maintaining low operating costs, notably by using free, open-source solutions (as opposed to traditional industry giants).

This report details the key stages of this development, the obstacles overcome, and the solutions provided to ensure a robust and efficient platform. It highlights the technical and strategic aspects of the project while emphasizing the benefits brought to local communities and businesses by our energy monitoring solution.

Cette page est laissée blanche intentionnellement.

Remerciements

Je tiens tout d'abord à exprimer ma profonde gratitude à M. Philippe Roose pour avoir proposé cette offre d'alternance aux étudiants, me permettant ainsi de postuler dans cette entreprise. Bien que le domaine de l'optimisation énergétique ne m'était pas étranger, les applications de l'informatique à ces besoins métiers spécifiques ont été extrêmement intéressantes à explorer. Cette expérience a considérablement élargi mes compétences, rendant mon profil professionnel encore plus diversifié.

Je remercie également M. Vincent Agi, mon tuteur d'alternance, pour son soutien constant et son implication dans mon apprentissage. Sa disponibilité, sa bienveillance et ses efforts à mon égard, sans oublier les nombreuses chocolaines apportées pour le déjeuner, ont été d'une grande aide. Ensemble, nous avons formé une équipe soudée, réussissant à accomplir de très belles choses dans les délais impartis grâce à une forte cohésion. Je lui suis reconnaissant pour son dévouement et des conseils qui ne pourraient être meilleurs, fondés sur une riche expérience.

Un grand merci à Guillaume, porteur du projet Invisio, pour la confiance qu'il m'a accordée à maintes reprises. Ses précisions sur les attentes clients et sa gestion d'entreprise agréable et presque familiale ont été précieuses. Merci également à Pierre, dont les connaissances métier ont été un véritable complément. Sa réactivité face à nos questions et sa bonne humeur constante ont grandement facilité notre travail.

Je tiens à remercier l'équipe de Gallium Ingénierie pour leur contribution à l'ambiance chaleureuse au sein de l'entreprise et pour leur accueil bienveillant.

Enfin, je remercie mon ami Nicolas, compagnon du permis moto, à qui j'avais promis une place dans ces remerciements. Son aide précieuse sur l'architecture de notre application et les solutions techniques avancées dans le domaine des communications réseau et de la conteneurisation a été inestimable. Jamais sans manquer une pointe d'humour concernant mes compétences limitées dans ce domaine, qui ont toujours rendu cette collaboration, entre étudiants, et amis, encore plus agréable.

Merci à tous pour cette expérience enrichissante.

Table des matières

1	Introduction	7
1.1	Motivation personnelle	7
1.2	Présentation d’InviséO et du Groupe Gallium	8
1.2.1	Activités et Expertise	8
1.2.2	Création et Mission d’InviséO	8
1.2.3	Services et Produits d’InviséO	8
1.3	Organisation de l’entreprise	9
1.4	Environnement de l’alternance	10
1.5	Déroulement de la première phase d’alternance	10
1.5.1	Présentation sommaire de PyScada	11
1.5.2	Les travaux de InviséO	11
1.5.3	La décision de InviséO vis-à-vis de PyScada	13
1.5.4	Missions de l’alternant	14
2	Production	15
2.1	Analyse du besoin	15
2.2	Les technologies retenues	15
2.3	L’architecture conçue	17
2.3.1	Les sources de données	18
2.3.2	La logique derrière l’application	19
2.3.3	La ligne de vie du logiciel	19
2.3.4	Détail des InviséO Box	20
2.3.4.1	Contraintes	20
2.3.4.2	Fonctionnement général	21
2.3.4.3	Le problème des ObjectId() de MongoDB	21
2.3.4.4	Paramétrage	21
2.3.4.5	Protocoles industriels supportés	22
2.3.4.6	Fonctionnement algorithmique	22
2.3.4.7	Conteneurisation des Workers	22
2.4	L’implémentation des DJUs dans l’API	24
2.5	Production de documentation	25
3	Bilans	27
3.1	Bilan général de l’alternance	27
3.2	Compétences du B.U.T.	28
3.3	Perspectives professionnelles	29
3.4	Bilan personnel sur mon parcours en B.U.T.	29
	Références	31

Table des figures

1	Situation géographique de InviséO	9
2	Organigramme de InviséO au 10 juin 2024	9
3	« Page d'accueil » de la solution InviséO avec PyScada	12
4	« Page 2 » de la solution InviséO avec PyScada	13
5	Architecture de la solution InviséO	18
6	Implémentation du motif MVC dans la solution InviséO	20
7	Fichier de configuration des <i>Workers</i>	23
8	Les DJUs et la consommation en fonction du jour dans les locaux de InviséO, visualisés grâce à la solution InviséO	25
9	Partie du README (backend) détaillant l'architecture du serveur	26
10	Page de Swagger affichant la liste des routes de l'API	26
11	Page de Swagger affichant les paramètres d'entrée et résultats de sortie de la route /api/auth/login	27

Liste des tableaux

1	Liste des membres de chaque partie	10
---	--	----

1 Introduction

1.1 Motivation personnelle

L’alternance représente pour moi une opportunité précieuse de gagner en compétences pratiques et de confronter mes connaissances théoriques à des situations réelles. Mon stage de deuxième année au sein du laboratoire de recherche de Domolandes m’a familiarisé avec des problématiques liées à la consommation d’énergie, éveillant ainsi mon intérêt pour ce domaine. Cette expérience m’a incité à approfondir mes connaissances en rejoignant InviséO, situé dans le même technopôle.

Travailler chez InviséO m’a permis d’aborder la thématique de la consommation d’énergie sous un angle différent, notamment à travers le prisme du développement logiciel. Cette nouvelle perspective est enrichissante et essentielle pour diversifier mon expertise et mieux comprendre les multiples facettes de la gestion énergétique. Cette alternance constituerait également pour moi une première expérience significative dans le développement logiciel, un domaine que je souhaite maîtriser pour devenir un professionnel polyvalent et compétent.

En choisissant cette entreprise, j’ai vu l’opportunité de conjuguer mon intérêt pour les technologies durables avec le développement de compétences techniques en programmation. Cette combinaison me permettra d’apporter une valeur ajoutée à mon futur parcours professionnel tout en contribuant à des projets innovants et durables.

Pour conclure, je tiens à préciser que ce rapport inclut des détails techniques que je juge nécessaires pour refléter la complexité réelle du domaine de l’optimisation énergétique et de son application informatique. Par ailleurs, je savais que la large étendue du domaine traité (l’informatique, le bâtiment, l’énergétique...) allait me permettre de rester dans une dynamique d’apprentissage tout au long de cette année, et allait venir constamment solliciter une réflexion qui m’est cruciale.

1.2 Présentation d'InviséO et du Groupe Gallium

InviséO est une entreprise dynamique spécialisée dans le suivi énergétique des bâtiments, née de l'idée novatrice de Guillaume Maillen. En complément de l'activité de **Gallium Ingénierie**, un bureau d'études fluides et thermiques, Guillaume Maillen a décidé de lancer le **Groupe Gallium**, regroupant ainsi les deux entités : **Gallium Ingénierie** et **InviséO**.

Gallium Ingénierie, fondée en 2019 par Guillaume Maillen, est un bureau d'études spécialisé dans les domaines des fluides et thermiques. Initialement créée en tant que micro-entreprise, elle a évolué vers le statut de SASU (Société par Actions Simplifiées Unipersonnelle) en 2020, grâce à un investissement en capital de 10 000 euros, facilitant ainsi ses démarches administratives.

1.2.1 Activités et Expertise

Gallium Ingénierie se spécialise principalement dans l'audit énergétique des bâtiments. Son expertise technique permet d'analyser et d'optimiser la performance énergétique des constructions. Ses services incluent :

- **Audit énergétique** : Évaluation détaillée des consommations énergétiques et identification des améliorations possibles.
- **Surveillance des données de consommation** : Suivi régulier pour garantir l'efficacité énergétique continue.
- **Proposition de solutions d'amélioration** : Recommandations techniques pour réduire les consommations.
- **Accompagnement dans la mise en œuvre** : Assistance durant toute la phase d'implémentation des mesures d'efficacité énergétique.

1.2.2 Création et Mission d'InviséO

InviséO, officiellement créée en février 2023, a vu ses activités de recherche et développement initialement réalisées sous le nom de Gallium. L'entreprise se concentre sur le suivi énergétique et propose des solutions innovantes pour améliorer l'efficacité énergétique des bâtiments. Parmi ses premiers clients, on compte des sites pilotes comme la MACS (Maremne Adour Côte-Sud) et les Chanvres de l'Atlantique, qui permettent à InviséO de tester et développer ses solutions.

1.2.3 Services et Produits d'InviséO

InviséO offre une gamme complète de services et produits pour la gestion énergétique :

- **Conception et installation de systèmes de télémesure** : Développement de capteurs et boîtiers de communication sur mesure pour une mesure précise de la consommation d'énergie.
- **Fabrication de capteurs et boîtiers de communication** : Dispositifs de pointe pour le suivi énergétique en temps réel.
- **Service de traitement et d'analyse des données** : Initialement, InviséO utilisait le logiciel Pyscada. Cependant, ce logiciel ne permettait pas de réaliser une solution distribuée multi-sites et s'est révélé trop complexe. En conséquence, nous avons décidé de redévelopper une solution logicielle de A à Z, mieux adaptée à nos besoins spécifiques.

En résumé, le **Groupe Gallium**, à travers **Gallium Ingénierie** et **InviséO**, propose une expertise pointue et des solutions complètes pour l'optimisation énergétique des bâtiments, alliant audit technique, suivi en temps réel, et analyse approfondie des données pour un avenir plus durable.

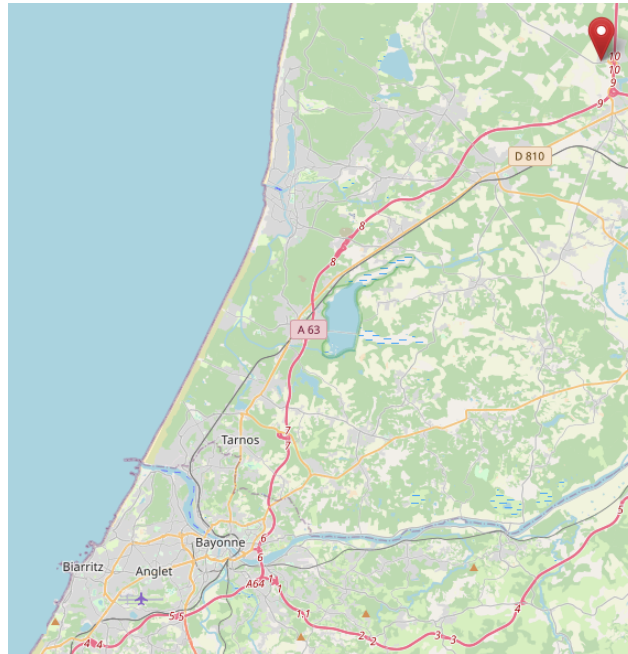


FIGURE 1 – Situation géographique de Inviséo

1.3 Organisation de l'entreprise

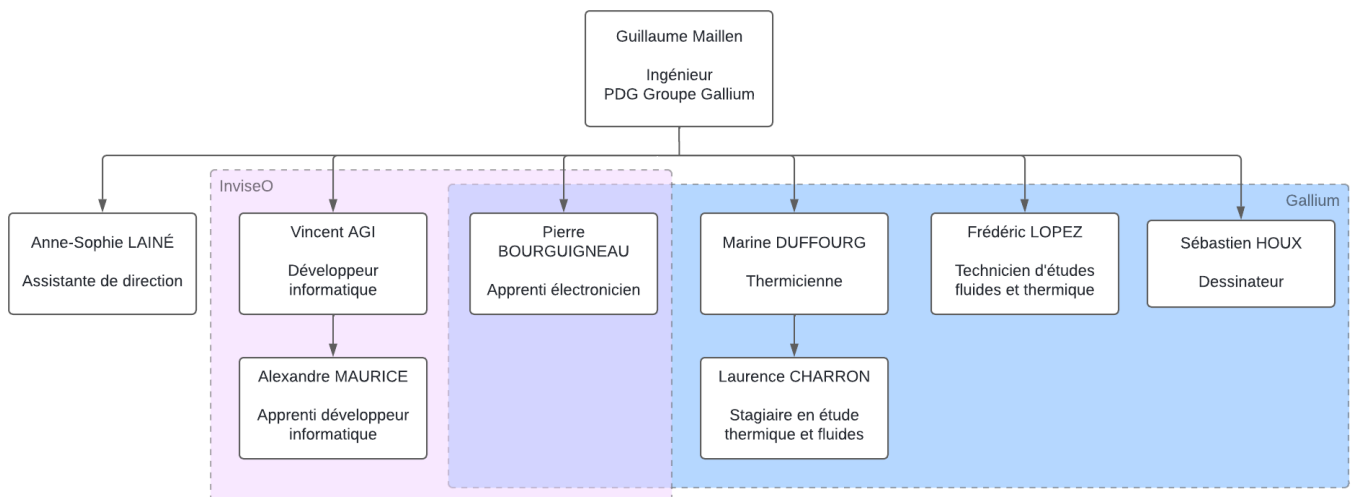


FIGURE 2 – Organigramme de Inviséo au 10 juin 2024

Je tiens à préciser que dans l’organigramme, Guillaume ne figure ni dans la partie « InviséO » ni « Gallium » (respectivement en rose et bleu) pour des raisons de simplifications schématiques, mais il participe évidemment bel et bien à leur activités, bien qu’il ne soit plus nécessairement directement présent dans toutes les actions métier.

1.4 Environnement de l’alternance

Mon alternance se déroule dans les locaux du Groupe Gallium, au sein du technopôle de Domolandes. InviséO démarrant à l’image d’une « startup », nous sommes alors une toute petite équipe, de quatre personnes, collaborant activement pour respecter des contraintes de temps compétitives, face à une concurrence de géants bien établis, qui tentent eux aussi de s’adapter aux nouvelles contraintes légales, pour convaincre une clientèle souhaitant une certaine qualité logicielle, et à moindre coût.

Dans le tableau 1, j’ai décidé de mettre en vert clair les personnes participant toujours activement au projet InviséO, et en rouge clair les personnes (chercheurs de l’UPPA) s’étant retiré du projet, les raisons de ce désistement seront précisées plus tard, dans la section 1.5.

Entité	Membre	Rôle
InviséO	M. Vincent AGI	Chef de projet et développeur
	M. Alexandre Maurice	Développeur en alternance
	M. Pierre Bourguigneau	Électronicien en alternance
	M. Guillaume Maillen	PDG de InviséO
UPPA	M. Camille Lavayssière	Enseignant-chercheur et développeur de PyScada
	M. Benoit Larroque	Enseignant-chercheur

TABLE 1 – Liste des membres de chaque partie

1.5 Déroulement de la première phase d’alternance

Ce que j’appelle ici la « première phase d’alternance » correspond à la période entre septembre 2023 et janvier 2024, période pendant laquelle le rythme de la formation alterne encore fréquemment, à raison d’environ 3 semaines en cours, et 3 semaines à l’IUT.

En septembre, au début de mon alternance, la solution logicielle développée était censée l’être en collaboration avec l’UPPA (l’Université de Pau et des Pays de l’Adour), et notamment deux enseignants chercheurs du laboratoire SIAME (Sciences pour l’Ingénieur Appliquées à la Mécanique et au génie Électrique), M. Camille Lavayssière et M. Benoit Larroque, dans le cadre d’une convention établie entre la MACS (communauté des communes de Maremne Adour Côte-Sud), l’UPPA, et InviséO. L’idée était de proposer à la MACS une solution de monitoring personnalisée, pour les nombreux bâtiments qu’ils avaient à nous fournir. InviséO a donc développé, de mai 2023 à février 2024, en collaboration avec l’UPPA. L’intérêt pour les universitaires était de continuer à pouvoir développer leur brique logicielle, et celui de InviséO était de pouvoir se lancer dans un domaine innovant et très concurrentiel, sans partir de zéro, en termes de connaissances techniques et de développement logiciel.

1.5.1 Présentation sommaire de PyScada

PyScada est un logiciel développé à l'origine par M. Martin Schröder, chercheur allemand à l'Université Technique de Berlin. PyScada se définit simplement comme :

Un système SCADA open source avec une IHM HTML5, construit à l'aide du *framework* Django¹.

En détails :

- Un système « SCADA », acronyme de « Supervisory Control and Data Acquisition », est en fait un *système de télégestion à grande échelle permettant de traiter en temps réel un grand nombre de télémesures et de contrôler à distance des installations techniques*².
- Un système déployé en *open source*, ce qui veut dire que le code source est librement accessible, modifiable et redistribuable par toute personne intéressée.
- Basé sur le *framework* Django, (considérez un *framework* comme une sorte de boîte à outils géante proposant des fonctionnalités prêtes à l'emploi pour créer des applications de manière simplifiée) lui-même basé sur le langage de programmation Python, Django vise à proposer aux développeurs une expérience orientée sur la réalisation de leur application « sans avoir à réinventer la roue »³.

Le développement de PyScada a débuté en 2013, et a maintenant été repris par M. Lavayssière, d'abord dans le cadre de sa thèse soutenue en fin 2022 (Lavayssière (2022)), puis dans la suite de ses recherches au sein du laboratoire SIAME. C'est cette brique logicielle qui était censée être utilisée pour Invisio aussi, dans le cadre d'un partenariat d'une plus longue durée.

La reprise de PyScada par M. Lavayssière s'inscrivait dans le cadre d'un projet de thèse du laboratoire visant à proposer à fournir aux étudiants du Diplôme Universitaire de Technologie (DUT) du parcours Génie Industriel et Maintenance (GIM), remplacé depuis 2021 par le Bachelor Universitaire de Technologie (BUT), un accès à des instruments permettant de réaliser à distance des travaux pratiques en électronique.

1.5.2 Les travaux de Invisio

Invisio (particulièrement Vincent et Pierre, alors présents dès la signature du contrat avec l'UPPA) a alors commencé le développement sur PyScada. Personnellement, j'ai aussi, de septembre à janvier, pendant cette « première phase d'alternance », contribué au développement de PyScada (développement de nouveaux plugins pour le logiciel, amélioration de fonctionnalités, correction de bugs...). Concrètement, et en parfaite transparence, le développement incrémental de PyScada s'est révélé extrêmement fastidieux, en raison de la structure de l'application, très lourde et peu documentée. Ce retard par rapport à la documentation est principalement dû à cause des raisons de la naissance de ce projet (expliquées ci-dessus), et à un manque de rigueur général dès le démarrage du projet. Aussi, PyScada n'était pas prévu pour correspondre aux besoins de Invisio, ne permettant pas le multisites grâce à une architecture distribuée⁴, et était à l'origine prévu pour avoir le niveau d'abstraction « le plus grand possible », et donc à rendre l'application la plus générique possible. La grande force de PyScada était sa compatibilité multi-protocoles industriels, dont beaucoup était

1. <https://github.com/pyscada/PyScada>

2. https://fr.wikipedia.org/w/index.php?title=Syst%C3%A8me_de_contr%C3%B4le_et_d%27acquisition_de_donn%C3%A9es&oldid=210182669

3. <https://www.djangoproject.com/>

4. https://fr.wikipedia.org/w/index.php?title=Architecture_distribu%C3%A9e&oldid=212787328

primordiaux pour Invisio (Webservice et Modbus principalement pour des solutions abordables en terme de temps et de coûts).

Pendant cette période, j'ai aussi réalisé des tâches annexes pour le logiciel interne de Gallium Ingénierie, *Gallium Software* (Authentification à deux facteurs, aspect global de l'application...), mais cette activité de a été extrêmement marginale, et il faut faire une distinction très claire entre ces tâches et la contribution au développement de PyScada, ce sont deux logiciels complètement différents. Aussi, je considère que les tâches réalisées pour le logiciel interne sont trop minimes pour pouvoir être valorisées dans un rapport, et l'utilité réelle de ces actions ne m'est pas parvenue non plus.

Finalement, durant cette période, le développement d'Invisio a été très largement en deçà de ce qui aurait été satisfaisant pour l'entreprise, et les résultats furent relativement peu tangibles et intéressants. Il m'est difficile de rendre compte de quoi que ce soit, d'un point de vue informatique, mon domaine d'activité, puisque l'écrasante majorité du temps a été passée à la configuration de pages, ainsi qu'à la recherche d'informations (documentation) concernant le logiciel. La correction de bugs et une méthodologie de développement s'approchant plus du *die-and-retry* (mécanisme issu du monde du jeu-vidéo) que d'une réelle méthode de développement d'une équipe de deux informaticiens de formation s'est aussi révélé être source d'énormément de frustration de ma part, ainsi que pour mon tuteur de stage, Vincent.

Je joins tout de même des captures des deux vues les plus avancées réalisées avec PyScada, pour Invisio, pour notre client.

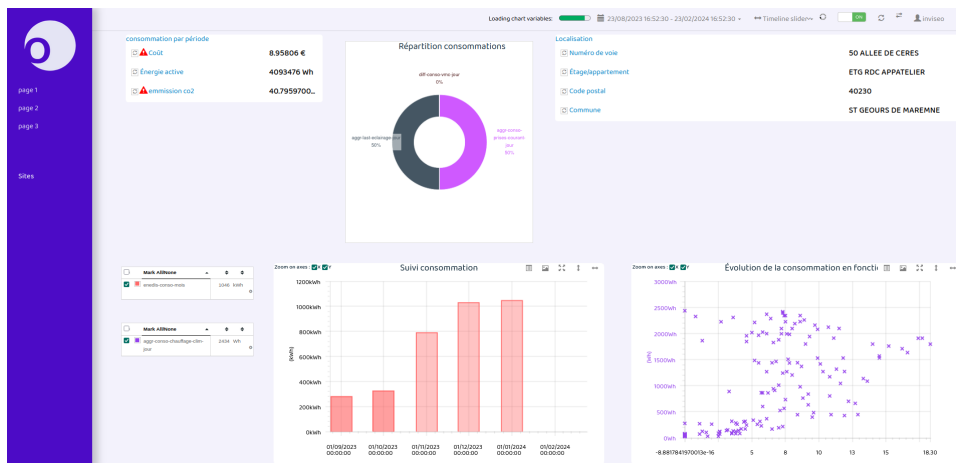


FIGURE 3 – « Page d'accueil » de la solution Invisio avec PyScada

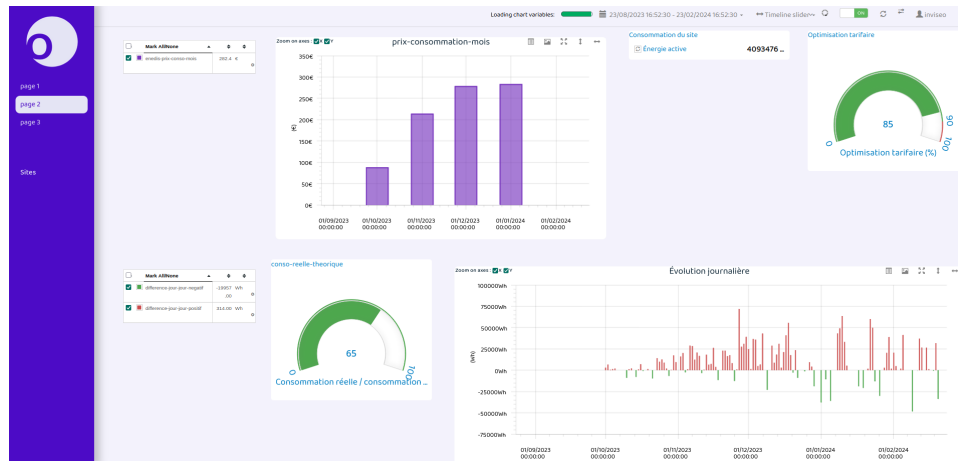


FIGURE 4 – « Page 2 » de la solution InvisioO avec PyScada

1.5.3 La décision de InvisioO vis-à-vis de PyScada

Au vu d'une échéance contractuelle demandée par notre client, arrivant à très grands pas, et de la très maigre progression réalisée par l'équipe de développement (tant du côté de l'UPPA que du côté d'InvisioO), il a paru évident que poursuivre avec PyScada n'allait pas être envisageable, pour le succès de InvisioO vis à vis de notre client, mais aussi pour le développement d'une solution pérenne.

C'est alors en février 2024 que Guillaume décide, au nom d'InvisioO, de stopper complètement l'aide au développement de InvisioO sur PyScada, au profit du développement intégral d'une nouvelle solution entièrement réalisée par InvisioO, qui aurait alors pour objectif direct de répondre dans un premier temps à la demande de notre client, la communauté des communes MACS, mais aussi de la solution qui permettrait à InvisioO par la suite de continuer son activité. Pour prendre cette décision, Guillaume s'est principalement basé sur le verdict que Vincent, Pierre et moi avons eu à donner, à la suite de ces 6 mois d'utilisation de PyScada, mais aussi par les contraintes de temps extrêmes qui nous étaient imposées.

Avec le recul, l'apport de l'UPPA dans le projet a tout de même permis à InvisioO de devenir plus indépendant et agile concernant des concepts liés à l'ingénierie du bâtiment et des communications (particulièrement le protocole industriel Modbus), nous donnant aussi des exemples de méthodes d'affichages et indicateurs importants pour l'étude de la consommation énergétique.

Enfin, la suite de ce rapport sera exclusivement (ou presque) dédiée à la présentation de la solution logicielle InvisioO, du choix des technologies et de leur architecture au déploiement final du logiciel, ainsi que les programmes légers qui lui sont associés et complémentaires.

1.5.4 Missions de l'alternant

Suite à un changement de direction concernant l'outil développé pour la solution Invisio, mon rôle a été redéfini pour se concentrer sur un objectif précis : participer au développement d'une application web responsive répondant aux besoins spécifiques de Invisio.

Dans ce cadre, mes missions principales ont inclus :

1. **Développement backend** : Participation active à la conception et à l'implémentation du backend de l'application. Cela comprenait le développement des API, la gestion de la base de données, et l'intégration de services tiers. Mon expertise en backend m'a permis de contribuer significativement à l'optimisation des performances et à la sécurisation des données.
2. **Conception de l'architecture** : Collaboration avec l'équipe de développement pour définir l'architecture de l'application, en s'assurant qu'elle soit scalable et maintenable. J'ai aidé à choisir les technologies et les *frameworks* appropriés pour le backend, tels que Node.js et Express, ainsi qu'à établir les bonnes pratiques de codage.
3. **Résolution de problèmes techniques** : Identification et résolution des problèmes techniques liés au backend. Cela impliquait la détection des bugs, l'optimisation des requêtes SQL, et l'amélioration de la gestion des erreurs pour garantir une application stable et fiable.
4. **Collaboration backend/frontend** : Travailler en étroite collaboration avec Vincent, s'occupant alors lui principalement de la partie frontale (UI/UX, User Interface/User Experience) pour garantir une intégration harmonieuse entre le frontend et le backend, et pour s'assurer que l'application finale offre une expérience utilisateur fluide et intuitive.
5. **Développement de la Invisio Box** : Conception et implémentation d'un client ultra léger installé sur une machine physique. Ce logiciel permet d'interroger les dispositifs (compteurs, capteurs) installés chez nos clients, et ce en incluant une compatibilité multiprotocoles et évolutive, sans nécessiter l'action d'un technicien sur les machines (mise à jour automatique du logiciel et des mesures à monitorer)
6. **Rédaction de documentation** : Création et mise à jour de la documentation technique complète de l'application. Cela incluait la rédaction des manuels de l'API, des guides de déploiement et de maintenance (en mode « développement » pour le backend et le frontend, mais aussi pour la mise en production de l'application globale), ainsi que des guides de démarrage pour les potentiels futurs développeurs de l'application.

2 Production

2.1 Analyse du besoin

Comme énoncé précédemment, l'entreprise InvisioO travaille dans le but de répondre au décret BACS. Ce décret vise à améliorer l'efficacité énergétique des bâtiments en automatisant et en contrôlant les systèmes de Gestion Technique des Bâtiments (GTB).

Pour se conformer à cette réglementation, notre application doit répondre à plusieurs besoins essentiels :

- **Surveillance et contrôle en temps réel** : Permettre la surveillance continue des systèmes de gestion technique des bâtiments afin d'identifier et de corriger rapidement toute défaillance ou inefficacité. Le « temps-réel » correspond en fait ici, et pour le moment, à un pas de temps « horaire » régulier.
- **Optimisation de l'efficacité énergétique** : Intégrer des fonctionnalités permettant d'optimiser la consommation d'énergie, en ajustant automatiquement les systèmes de chauffage, de ventilation et de climatisation (CVC) en fonction de l'occupation et des conditions environnementales.
- **Conformité réglementaire** : Assurer que toutes les actions et les systèmes mis en place respectent les exigences du décret BACS, notamment en termes de collecte et de reporting des données énergétiques.
- **Facilité d'utilisation et d'intégration** : Offrir une interface utilisateur intuitive et la capacité de s'intégrer facilement avec les systèmes existants dans les bâtiments pour minimiser les disruptions et les coûts de mise en œuvre.
- **Analyse des données et rapports** : Fournir des outils d'analyse avancés pour traiter les données collectées et générer des rapports détaillés sur la performance énergétique, facilitant ainsi la prise de décision pour les gestionnaires de bâtiments. Ainsi, Gallium Ingénierie pourra fournir une analyse basée sur des données fiables récupérées *in situ*, et les soumettre aux clients du Groupe pour proposer des améliorations énergétiques.
- **Application légère chez le client** : les données de consommation qui relèvent du sous-comptage, une implémentation physique de **compteurs** relevant de manière plus précises les consommations relatives à un ou des appareil(s) (comme un radiateur électrique, les prises de courant, la ventilation...), ainsi que les données remontées par des **capteurs** liés au confort (température, pourcentage d'humidité, taux de CO₂...) doivent elles être remontées par une machine physiquement installée sur le réseau de notre client, pour pouvoir communiquer avec ces instruments. Le logiciel installé sur ces machines (appelées pour nos clients des « InvisioO Boxes ») doit aussi être développé par InvisioO.

2.2 Les technologies retenues

Après une longue concertation au sein de notre petite équipe de développement, Vincent et moi avons décidé de lancer le logiciel InvisioO sur une « *stack MEAN* ». Le terme « *stack* » fait ici référence à une « pile », soit un ensemble de plusieurs technologies informatiques. Le sigle « *MEAN* », lui, fait référence aux technologies bien connues et documentées « MongoDB, Express, Angular, NodeJS ». Cette pile informatique est largement connue à travers le monde, éprouvée, et utilisée par de nombreux géants de l'industrie souhaitant utiliser l'informatique libre. Aussi, elle est de nos jours favorisée par rapport à des piles plus traditionnelles comme la pile « LAMP » (Linux, Apache, MySQL, et traditionnellement PHP), comme par exemple étudié dans Dunka et al. (2018).

Nous avons alors besoin d'un lancement très rapide pour le renouveau de notre solution. Pour

cela, rien de mieux que des *frameworks* en JavaScript, un langage extrêmement permissif et largement répandu, qui permettait de réaliser les premières étapes du logiciel sans accroc. Aussi, JavaScript est un langage relativement intuitif, et maîtrisé par un très grand nombre d'informaticiens à travers le monde, en faisant probablement un bon choix pour la maintenabilité du logiciel, si jamais d'autres alternants ou nouveaux développeurs venaient à travailler sur le projet. Finalement, cette solution est extrêmement simple à déployer, de manière universelle, et sur grand nombre de systèmes, en plus de s'améliorer continuellement au fur et à mesure des années, augmentant ainsi la charge que peut ingérer les logiciels en JavaScript, mais également en promettant des résultats qui en deviennent, avec les années, presque bluffants (JavaScript arrive maintenant à des performances relativement incroyables comparé à ce qu'il était il y a de ça une décennie, et se place maintenant presque en tête des langages informatiques les plus efficaces)⁵.

Voici une présentation sommaire de chacune des technologies de la *stack MEAN*, ainsi que les raisons qui nous ont poussées à choisir une telle stack pour le développement de la solution InviséO :

- **MongoDB**⁶ : Une base de données NoSQL orientée documents, qui permet de stocker des données sous forme de documents JSON. Elle est particulièrement adaptée pour les applications nécessitant une grande flexibilité et évolutivité. Nous avons longtemps hésité à choisir une base de données plus traditionnelle (et par conséquent relationnelle), mais avons finalement décidé d'opter pour l'ultra-flexibilité avec MongoDB, choix que nous ne regrettons absolument pas, à la date de la rédaction de ce rapport. Nous avons déjà pu constater une grande efficacité et optimisation lors de la création de nos schémas, eux aussi très divers et variés (l'objet peut changer de forme en fonction du protocole, puis du type du protocole, puis du format de l'adresse que nous devons interroger par exemple).
- **Express**⁷ : Un *framework* web minimaliste pour Node.js, utilisé pour construire des applications web et des API. Express simplifie la gestion des requêtes et des réponses HTTP, facilitant ainsi le développement de serveurs web robustes. Extrêmement bien documenté, et largement éprouvé, ce framework s'est imposé. Nous avons aussi hésité avec .NET, *framework* basé sur le langage de programmation compilé « C# », compilé, et développé par Microsoft. Finalement, *Gallium Software* ainsi que de nombreux autres projets développés par Vincent étaient réalisés en .NET, et nous avons souhaité changer, apportant une nouveauté de plus à InviséO, et renforçant la cohérence de JavaScript dans le projet.
- **Angular**⁸ : Un *framework* frontend développé par Google, utilisé pour créer des applications web dynamiques et interactives. Angular permet de structurer le code de manière modulaire et d'améliorer la maintenabilité de l'application. J'ai déjà eu l'occasion de travailler avec Angular par le passé, mais j'ai souhaité n'avoir que peu d'implication dans le développement du frontend, de par mon appétence relativement faible pour cet aspect de l'informatique. Ainsi, j'ai tout de même été favorable à son utilisation, puisque je saurai intervenir si besoin, mais la main a été largement laissée à Vincent pour le développement de la partie frontale, séparant en quelque sorte notre petite équipe en deux groupes, avec une séparation des tâches claires.
- **Node.js**⁹ : Un environnement d'exécution JavaScript côté serveur, qui permet de construire des applications réseau rapides et évolutives. Node.js utilise un modèle d'E/S non-bloquant et événementiel, ce qui le rend efficace pour les applications en temps réel. De plus, la documentation de Node.js est extrêmement fournie, et c'est un environnement d'exécution très

5. https://en.wikipedia.org/wiki/Comparison_of_programming_languages

6. <https://www.mongodb.com/>

7. <https://expressjs.com/>

8. <https://angular.dev/>

9. <https://nodejs.org/fr>

largement utilisé à travers le monde. Son déploiement est simple et peut être conteneurisé très facilement, en une seule ligne de commande.

2.3 L'architecture conçue

Très rapidement, la solution s'était imposée pour le service proposé par Invisio. Ce logiciel fonctionnant sous forme de SaaS (« *Software as a Service* »), il nous a paru évident que l'architecture devrait reposer sous la forme d'une API REST.

Qu'est-ce qu'une API ? Une API, ou « *Application Programming Interface* », est un ensemble de règles et de protocoles qui permet à nos applications de communiquer entre elles. Il faut voir une API comme un serveur dans un restaurant : vous énoncez une demande (votre commande) au serveur, qui la transmet ensuite à la cuisine (le système ou service). Une fois la commande prête, le serveur vous apporte le plat. De la même manière, une API permet à une application de demander des informations ou des services à une autre application, et de recevoir une réponse structurée. Cela facilite l'intégration et l'interaction entre différents logiciels.

Mais alors qu'est-ce qu'une API « REST » ? Une API REST, ou « *Representational State Transfer* », est un moyen de communication entre différents logiciels via le web. Elle permet à notre application de dialoguer avec d'autres services ou applications en utilisant des requêtes HTTP standardisées, comme GET, POST, PUT et DELETE. En termes simples, une API REST est comme un messenger qui transporte des demandes et des réponses entre les différentes parties d'un logiciel.

Les avantages de l'API REST

- **Simplicité** : Les API REST utilisent les requêtes HTTP standard, ce qui les rend faciles à comprendre et à utiliser pour les développeurs.
- **Scalabilité** : Grâce à son architecture stateless (sans état), une API REST peut facilement évoluer pour gérer un grand nombre de requêtes simultanément.
- **Flexibilité** : Elle permet d'interagir avec différents types de clients, comme des applications web, mobiles ou autres services, sans nécessiter de modifications importantes de l'API.
- **Interopérabilité** : Les API REST peuvent être utilisées avec presque tous les langages de programmation, ce qui facilite leur intégration dans divers environnements technologiques.

Comme précisé ci-dessus, notre API sert alors de charnière entre les données (et notre Système de Gestion de Base de Données, SGBD) et l'affichage au client, soit le serveur frontal, qui lui met en ordre les données, et formate l'affichage, pour rendre au client une vue graphique et compréhensible des données (comme les sites web que nous consultons tous les jours).

Pour le moment, le serveur Web (Nginx), la vue (Angular), l'API (Express) et la base de données (MongoDB) ont été conteneurisés grâce à Docker. Docker est une plateforme permettant de créer, déployer et gérer des conteneurs, des unités logicielles empaquetant le code d'une application ainsi que toutes ses dépendances. L'immense force de Docker est d'assurer que l'application fonctionne de manière uniforme et cohérente dans différents environnements, que ce soit sur une machine locale, dans un centre de données ou dans le cloud. Cela assure donc la cohérence et la pérennité de notre application, en lui permettant de pouvoir gagner significativement en taille, de réduire au maximum les dépendances (la base de données ne dépend alors pas de l'API par exemple), mais aussi de pouvoir être déployée n'importe où, peu importe le système d'exploitation, sa version, l'environnement matériel. Aussi, un conteneur étant alors une réplique miniaturisée d'un système d'exploitation, il ne met pas en danger l'intégrité du fonctionnement de la machine sur lequel il est installé.

La figure 5 détaille de manière schématique les différentes composantes de l'application.

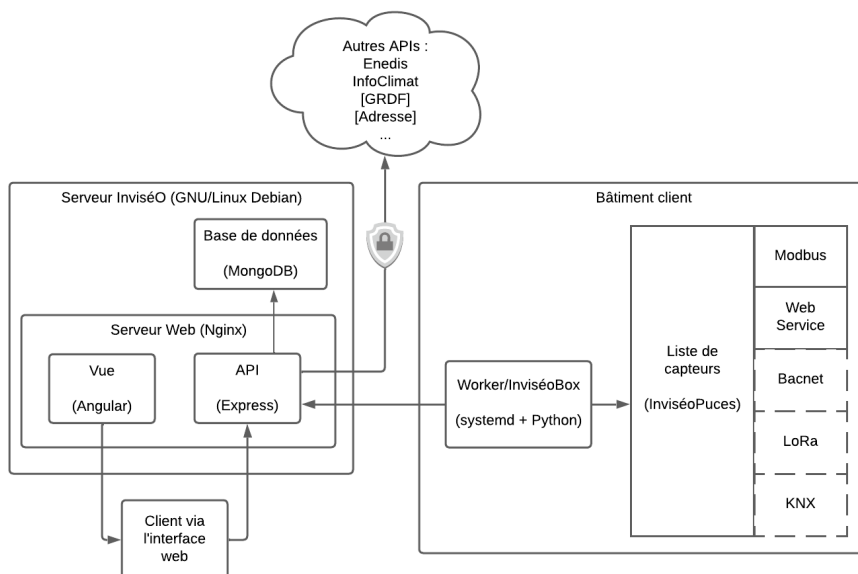


FIGURE 5 – Architecture de la solution Inviséo

2.3.1 Les sources de données

Notre application contient pour le moment quatre sources de données :

- Les données saisies par les utilisateurs (un bâtiment, son numéro de PDL pour récupérer les données de Enedis, ses dimensions... Un appareil de type capteur qui est capable de relever la température, l'humidité...), qui permettent au logiciel d'instancier les données que vont pouvoir consulter les utilisateurs.
- Les données d'Enedis, obtenues en réalisant une requête à notre accès privé à l'API « SGE Tiers » de Enedis¹⁰, et en donnant le numéro de PDL, identifiant d'un bâtiment, permettant à Enedis de renvoyer les données de consommation et de puissance du site en question.
- Les données issues de l'« Open Data météo » d'InfoClimat¹¹, permettant de récupérer les données de température **extérieure** d'un bâtiment, pour une zone géographique relativement proche. Par exemple, pour les locaux d'Inviséo, localisés à Saint-Geours-de-Mareme, la station météo d'InfoClimat la plus proche est celle de Magescq, à 10km au Nord. InfoClimat nous permet aussi de récupérer le DJU (degré jour unifié) d'un jour en particulier pour un bâtiment en particulier, information très importante pour étudier notamment les données de chauffage du bâtiment.
- Les données des Inviséo Boxes, véritable plus-value de Inviséo par rapport aux concurrents, qui permettent d'obtenir des données précises et *in situ*, concernant notamment du sous-comptage électrique (consommation et puissance) et des données relatives au confort des usagers du bâtiment.

10. <https://www.enedis.fr/acceder-aux-donnees-fournies-par-enedis>

11. <https://www.infoclimat.fr/opendata/>

Nous pourrions, plus tard, comme vous pouvez déjà le voir sur la figure 5, intégrer l'API « GRDF ADICT » de GRDF¹², pour récupérer aussi les données de gaz de nos clients, proposant alors un panel d'information et d'analyse plus large à nos clients. L'API « Adresse » proposée par le gouvernement¹³ pourrait aussi être intégrée pour faciliter la saisie des informations de géolocalisation des bâtiments de nos clients, et proposer, par exemple, une visualisation de son, ou de sa flotte de bâtiments grâce à OpenStreetMap¹⁴. Cette dernière intégration, bien qu'elle a déjà été évoquée, n'est pour l'instant pas prévue dans le logiciel.

2.3.2 La logique derrière l'application

La solution InvisoO utilise le motif d'architecture logicielle « MVC » (Modèle-vue-contrôleur). Le motif est composé de trois types de modules ayant trois responsabilités différentes : les modèles, les vues et les contrôleurs.

- Un modèle qui contient les données à afficher, les structurant de manière uniforme.
- Une vue qui contient la présentation de l'interface graphique.
- Un contrôleur qui contient la logique concernant les actions effectuées par l'utilisateur (création, lecture, suppression ou modification d'objets).

Le choix du modèle MVC permet de respecter plus facilement le principe de responsabilité unique des principes de conception SOLID largement répandus en informatique, notamment en morcelant la responsabilité des fonctions dans le programme¹⁵.

Dans notre choix des technologies, la vue est remplacée par Angular, puisqu'il s'agit d'un cadriciel (ou *framework*) exclusivement réservé à la partie frontale d'une application web. Le reste des rouages de l'application est géré par Express. Encore une fois, utiliser Angular permet de conserver une universalité au niveau des modèles, puisqu'un modèle servant dans la vue pour un formulaire de création d'un objet sera presque en tous points similaire à la structure de validation utilisé par le modèle afin d'assurer sa bonne gestion par le contrôleur.

Une illustration de la logique du motif MVC est présentée dans la figure 6 ci-dessous.

2.3.3 La ligne de vie du logiciel

Pour simplifier, voici comment cela fonctionne pour une instanciation d'un site (ou bâtiment) :

- L'utilisateur demande à consulter notre site web, <https://client.inviso.fr>.
- L'interface est renvoyée par la vue (serveur Web du conteneur Angular).
- L'utilisateur réalise une requête préformatée par la vue grâce à l'URL donnée par cette dernière (<https://client.inviso.fr/api>). Un exemple de requête serait une requête HTTP de type POST, pour créer un nouvel objet, vers l'URL <https://client.inviso.fr/api/sites/>, en mettant dans le *payload* (peut se traduire par « charge utile ») de son *body* (le corps de la requête HTTP) les spécifications de l'objet « site » qu'il souhaite créer.
- Le serveur va ensuite récupérer, à l'aide des informations de communication saisies par les utilisateurs (cet usage est en réalité réservé à l'expertise d'InvisoO), les données relevées par les InvisoO Box.
- Quand un utilisateur demandera à consulter les données d'un bâtiment, il fera alors une requête au backend, qui ira chercher les données dans les différentes sources de données (cf. 2.3.1) pour les envoyer au client. Le client bénéficiant de la vue, chargée dans son navigateur internet, pourra alors bénéficier d'un affichage graphique de ses données.

12. <https://sites.grdf.fr/web/portail-api-grdf-adict>

13. <https://adresse.data.gouv.fr/api-doc/adresse>

14. <https://www.openstreetmap.org/>

15. [https://fr.wikipedia.org/wiki/SOLID_\(informatique\)](https://fr.wikipedia.org/wiki/SOLID_(informatique))

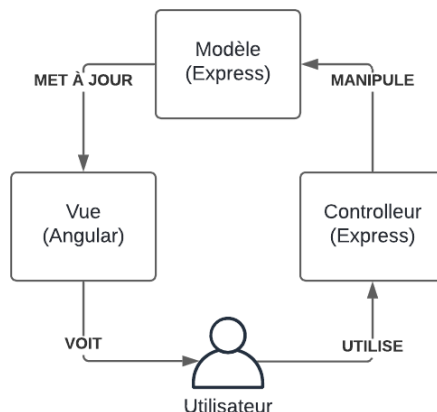


FIGURE 6 – Implémentation du motif MVC dans la solution Inviséo

2.3.4 Détail des Inviséo Box

Bien que j'aie principalement géré la logique côté backend, j'ai assumé de manière exclusive le développement et la conception des Inviséo Box. Je détaillerai donc cette section avec conviction et rigueur. Aussi, étant donné que « Inviséo Box » est un terme commercial, et je préfère me concentrer sur l'explication méthodologique du développement de ces boîtiers. Par conséquent, d'un point de vue rigoureux et strictement informatique, je les désignerai dorénavant comme des « *Workers* », représentant des appareils réalisant des tâches simples au sein d'un système auquel ils sont intégrés.

Les Workers sont de simples ordinateurs sur lesquels Inviséo vient installer un système d'exploitation (le cas échéant GNU/Linux Debian, à date la douzième version stable) qui vont interroger *le plus possible* les dispositifs de mesure installés chez le client. À intervalles réguliers (le cas échéant un pas de trente minutes pour correspondre au pas de temps horaire imposé par le décret BACS), ces ordinateurs vont envoyer les données mesurées sur les bâtiments à l'API, afin qu'elles puissent être consultées « en temps réel » par les utilisateurs (qui peuvent être des usagers du bâtiment).

2.3.4.1 Contraintes

La nouveauté dans la solution Inviséo est aussi de proposer une solution de monitoring énergétique non-intrusive, qui se veut la plus simple à installer, « clé en main », et surtout, qui fonctionne dans le plus de cas d'usages.

Cela voulait dire que nous ne devons pas utiliser de VPN (Virtual Private Network, une connexion permettant de se retrouver virtuellement dans un réseau privé distant) car cela aurait nécessité l'ouverture de ports¹⁶ sur le routeur du client, ou alors, si un VPN était déjà mis en place, d'utiliser une configuration spécifique pour se connecter au réseau.

Nous n'eûmes pas non plus la possibilité d'utiliser des boîtiers 4G, car ceux-ci auraient changé fréquemment en raison de leur adressage d'IP dynamique¹⁷, et que cela posait des problèmes de transparence pour nos clients. En effet, venir installer un réseau parasite chez un de nos clients n'était pas une solution viable ni morale, bien que certains auraient probablement acceptés. La mise

16. [https://fr.wikipedia.org/wiki/Port_\(logiciel\)](https://fr.wikipedia.org/wiki/Port_(logiciel))

17. https://fr.wikipedia.org/wiki/Adresse_IP

en place d'un tel dispositif aurait rendu nos clients à la merci d'attaques extérieures par le biais de notre appareil, consultant des données qui elles, étaient bel et bien chez eux.

2.3.4.2 Fonctionnement général

J'ai donc opté pour une logique complètement basée sur le système d'API de notre solution. Peut-être très intuitive au premier abord, je dois avouer que ce n'est pas la solution qui m'est immédiatement venue à l'esprit. Tout d'abord, comme prérequis, il nous faudra un appareil lié à Internet. Pour cela, rien de plus simple, il sera installé sur le réseau de nos clients, comme n'importe quel autre ordinateur.

Les « *Workers* » seront implémentés dans le code comme faisant partie d'un seul et unique « Site ». Lors de leur création, un « *token* » est généré. Le terme « *token* », bien que sémantiquement correct, ne fait pas vraiment référence à l'objet *token* au sens où on l'entend traditionnellement en informatique. En effet, un *token* est supposé être changé avec le temps, à cause d'une durée de validité prédéterminée par exemple. La différence ici est que le *token* généré sert en fait plutôt de moyen d'identification du *Worker*.

Ce dernier fera alors une requête vers l'URL « `https://client.inviso.fr/api/workers/devices?token=<token>` », en passant en paramètres l'attribut « *token* » pour obtenir la liste des appareils qu'il doit monitorer, ainsi que des informations sur le moyen de communication à employer pour les atteindre. Il réalisera ensuite les mesures, et les enverra au serveur décentralisé (l'API).

2.3.4.3 Le problème des ObjectId() de MongoDB

Précédemment, j'avais décidé d'utiliser les identifiants générés automatiquement par MongoDB, plutôt que de générer automatiquement un token unique plus complexe. Voici un exemple :

```
<ObjectId("5fa6a5f786dcbc4d3bf5e6e2")>
```

Le problème avec ce moyen d'authentification, c'est que les identifiants générés automatiquement par MongoDB sont séquentiels (« ...75 », « ...76 », « ...77 »...) ou contiennent des motifs prévisibles (une chaîne alpha-numérique de 24 caractères). Cela signifie qu'un attaquant pourrait potentiellement deviner ou essayer d'énumérer ces identifiants, ce qui pourrait permettre des accès non autorisés à des données sensibles, et potentiellement, dans notre cas, accéder à la configuration de nos appareils.

2.3.4.4 Paramétrage

Il faut ensuite paramétrer les *Workers*. Pour cela, une fois qu'on a rattaché le *Worker* à un site, la procédure est la suivante :

- Associer à ce *Worker* des appareils (modélisés comme des « *Devices* » dans le code), compteurs ou capteurs, avec leur protocole associé.

Exemple : **Modbus**, en mode **RTU** de type **RS-485**.

- Associer à ces appareils une ou plusieurs mesures (modélisées comme des « *Measurements* » dans le code) précises, contenant le chemin spécifique à aller interroger pour obtenir les informations qui nous intéressent. Ces chemins sont souvent issus de la documentation constructeur, puisque tous les protocoles industriels sont adaptés par chaque commerçant.

Exemple : Sur le port `/dev/serial/32`, avec **1 bit de stop**, et une taille de donnée correspondant à **8 bits**.

- Les données enregistrées, correspondant aux valeurs de la mesure à un instant T, correspondent alors à un champ (modélisés comme des « *Fields* » dans le code), détenant les informations relatives à la mesure.

Exemple : La **mesure X** a relevé **36Wh** dans les **30 dernières minutes**.

Finalement, avec toutes ces données, on peut remonter l'information pour obtenir quelque chose de compréhensible au premier coup d'œil par un utilisateur de notre application.

Exemple : La PAC (**Pompe À Chaleur**) a consommé **36Wh** dans les **30 dernières minutes**.

Autre exemple : L'**éclairage de la zone industrielle** a eu une puissance maximale de **12W** de **8h30 à 9h30**.

2.3.4.5 Protocoles industriels supportés

Pour le moment, la priorité était pour Invisio de pouvoir interroger les compteurs et capteurs que l'entreprise maîtrisait déjà. Parmi ceux-ci, on compte notamment :

Modbus : un protocole de communication série utilisé principalement dans les systèmes d'automatisation industrielle. Il permet la transmission de données entre des appareils électroniques via des réseaux de communication, facilitant ainsi le contrôle et la surveillance de divers équipements. Le protocole fonctionne sur des configurations maître-esclave, où le maître envoie des commandes et reçoit des réponses des esclaves. Pour le moment, nous utilisons les compteurs de la marque RS PRO, en mode RTU, et de type RS-485 (soit en port série). Nous utilisons la librairie PyModbus¹⁸, programmée en Python, pour simplifier l'interrogation de ces appareils. Le signal Modbus arrive dans les ports série de notre ordinateur sous Linux grâce à un convertisseur RS-485 vers USB, issu du marché. Notre *Worker* est donc capable de lire sans accroc les registres Modbus.

WebService : un protocole de communication qui permet l'échange de données sur un réseau, en l'occurrence le réseau local de notre client. Le *Worker* étant lui aussi sur le même réseau, il lui suffit d'accéder au contenu stocké sur l'adresse IP de notre capteur (exemple : `http://192.168.0.12`), pour obtenir un fichier au format JSON, très facilement lisible.

Voici un exemple de réponse de nos capteurs :

```
{
  "temp": 22.5,
  "hum": 45,
  "co2": 400
}
```

2.3.4.6 Fonctionnement algorithmique

Pour éviter de perdre des données si jamais le *Worker* n'est plus alimenté (panne de courant ou autre) pendant un bref (ou plus long) instant, alors les données sont en permanence écrites dans la base de données, et lors de l'envoi, sont aussi dupliquées dans un fichier JSON (ligne 3 de l'algorithme 1). Cela veut dire que le cas d'erreur est aussi géré si jamais c'est l'API, le serveur distant, qui tombe en panne, assurant ainsi une disponibilité maximale des données dès que le problème sera résolu.

2.3.4.7 Conteneurisation des Workers

Les *Workers* bénéficient eux aussi d'un programme complètement conteneurisé. Il était aussi très important, si jamais un changement au niveau des appareils physiques était fait, qu'il puisse

18. <https://github.com/pymodbus-dev/pymodbus>

Algorithme 1 Algorithme du programme principal des InvisoO Box

```

1: Charger les variables d'environnement (URL de l'API, token de l'InvisoO Box)
2: Vérifier l'intégrité des variables d'environnement
3: Initialiser les fichiers JSON
4: Initialiser la base de données SQLite
5: procedure BOUCLEPRINCIPALE
6:   while vrai do
7:     Charger la configuration des appareils à interroger via l'API
8:     for chaque dispositif do
9:       if protocole Modbus et mode RTU then
10:        Gérer le dispositif Modbus
11:       else if protocole Webservice then
12:        Gérer le dispositif Webservice
13:       end if
14:     end for
15:     Construire et envoyer les statuts des dispositifs
16:     Construire et envoyer les données de mesure
17:     Attendre l'intervalle défini
18:   end while
19: end procedure

```

se mettre à jour, automatiquement, pour avoir toujours la dernière version du logiciel, et ne jamais perdre la compatibilité avec les nouveaux types de protocoles qui pourraient être implémentés.

Aussi, la conteneurisation permet de lancer le programme automatiquement au démarrage de la machine ou bien de le relancer s'il plante pendant l'exécution. Cela est assuré par l'image « watchtower », présenté dans la figure 7. Cette implémentation a été cruciale lors du développement du programme destiné aux *Workers*, puisque de nombreux bugs affectaient le fil d'exécution du programme principal. Cela a permis notamment de déboguer, et nous rassure quand à l'*uptime* (ou disponibilité) constante de nos *Workers*.

```

1  services:
2    invisobox:
3      image: ghcr.io/inviso/invisobox:latest
4      container_name: invisobox
5      privileged: true
6      user: root
7      restart: always
8      pull_policy: always
9      environment:
10       - token=XXXXXXXXXXXXXXXXXXXX
11       - interval=1800
12       - url=XXXXXXXXXXXXXXXXXXXX
13
14    watchtower:
15      image: containrrr/watchtower:latest
16      restart: always
17      command: --interval 5 --debug --cleanup invisobox

```

FIGURE 7 – Fichier de configuration des *Workers*

2.4 L'implémentation des DJUs dans l'API

Comme le développement de notre application touche énormément de branches différentes, et que beaucoup de fonctionnalités ont dû être développées, j'ai préféré en détailler une implémentation plutôt que de commencer une longue énumération. J'ai donc choisi de présenter mes travaux concernant les DJUs.

Les DJUs, ou Degrés-Jours Unifiés, sont une mesure utilisée pour quantifier la demande énergétique nécessaire pour chauffer ou refroidir un bâtiment. Ils sont calculés en fonction des écarts de température par rapport à une température de référence, qui est définie à 18°C.

La formule simplifiée (que nous utilisons) pour calculer les DJUs est la suivante :

$$\text{DJU} = \max\{0, 18 - \frac{T_{\max} + T_{\min}}{2}\}$$

où $T_{\max}$ est la température maximale de la journée et $T_{\min}$ est la température minimale de la journée. Dans cette formule, la fonction $\max(a, b)$ renvoie le maximum entre a et b .

En d'autres termes, le DJU correspond à la différence entre la température de référence (18°C) et la température moyenne de la journée, lorsque cette différence est positive. Si la température moyenne de la journée est supérieure ou égale à 18°C, le DJU est de 0. Cela permet de quantifier la nécessité de chauffer (ou parfois de refroidir) un bâtiment en fonction des conditions climatiques.

Algorithme 2 Intégration des données météorologiques, dont les DJUs

```

1: Entrée : Site
2: Sortie : Mise à jour de la station météo avec les données complètes et renvoi du site
3: procedure INTÉGRATIONDES DonnéesMÉTÉO(Site)
4:   Étape 1 : Récupération de la station météo du site
5:   station ← récupérerStationMétéo(site)
6:   Étape 2 : Identification des jours manquants
7:   joursManquants ← identifierJoursManquants(station)
8:   Étape 3 : Récupération des données manquantes auprès d'InfoClimat
9:   donnéesMétéo ← récupérerDonnéesInfoClimat(joursManquants)
10:  Étape 4 : Mise à jour de la collection de la station météo avec les nouvelles données
11:  for jour in joursManquants do
12:    tempMax ← donnéesMétéo[jour].tempMax
13:    tempMin ← donnéesMétéo[jour].tempMin
14:    tempMoyenne ←  $\frac{\text{tempMax} + \text{tempMin}}{2}$ 
15:    DJU ←  $\max(0, 18 - \text{tempMoyenne})$ 
16:    insérerDonnées(station, jour(JJ/MM/AAAA)[tempMax,tempMin,tempMoyenne,DJU])
17:  end for
18: end procedure

```

L'algorithme 2 permet d'associer chaque jour une consommation de chauffage aux DJUs climatiques. Pour évaluer l'efficacité thermique du bâtiment, il suffit de comparer la courbe des DJUs à celle des consommations, comme montré dans la figure 8. Bien que les DJUs montrent une corrélation claire avec la consommation totale (les données détaillées étant partielles en raison du développement des Invisio Box), l'enveloppe thermique du bâtiment ne répond pas suffisamment à la température extérieure, indiquant la nécessité d'améliorations telles que le réglage du thermostat ou la configuration des horloges.

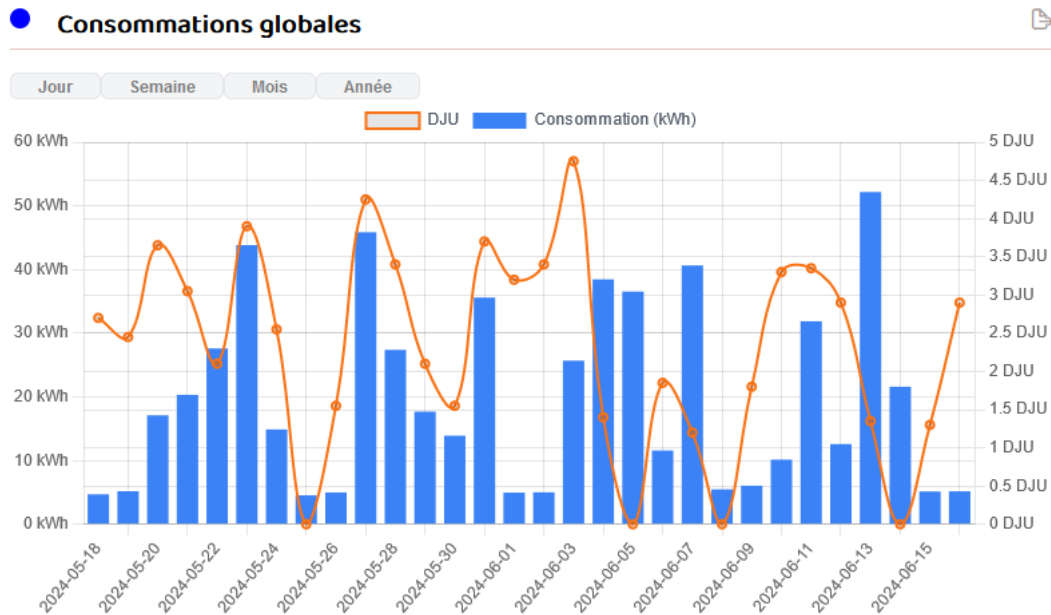


FIGURE 8 – Les DJUs et la consommation en fonction du jour dans les locaux de Inviséo, visualisés grâce à la solution Inviséo

2.5 Production de documentation

Comme mentionné précédemment, l'arrêt de l'utilisation de PyScada par Inviséo a en partie été causé par son manque de documentation. Nous avons fait alors, avec Vincent, un large travail de documentation concernant l'entièreté de l'application, dans sa globalité, puis dans le détail.

Alors que Vincent s'est principalement concentré sur la documentation du frontend ainsi que du fonctionnement de l'API Enedis (elle aussi très peu documentée), je me suis occupé de la documentation du backend (figure 9), via les README de Github, ainsi que grâce à Swagger, un langage de description d'interface permettant de décrire des API exprimées à l'aide de JSON. Swagger permet de générer automatiquement une documentation en ligne, consultable donc depuis notre logiciel (en mode « développement » uniquement cependant).

Cette documentation permettra aux prochains développeurs de prendre simplement connaissance de l'environnement de développement et de production, des bonnes pratiques de développement, ainsi que de l'architecture générale et du fonctionnement plutôt détaillé de certaines fonctionnalités dont l'usage peut parfois paraître occulte.



FIGURE 9 – Partie du README (backend) détaillant l'architecture du serveur

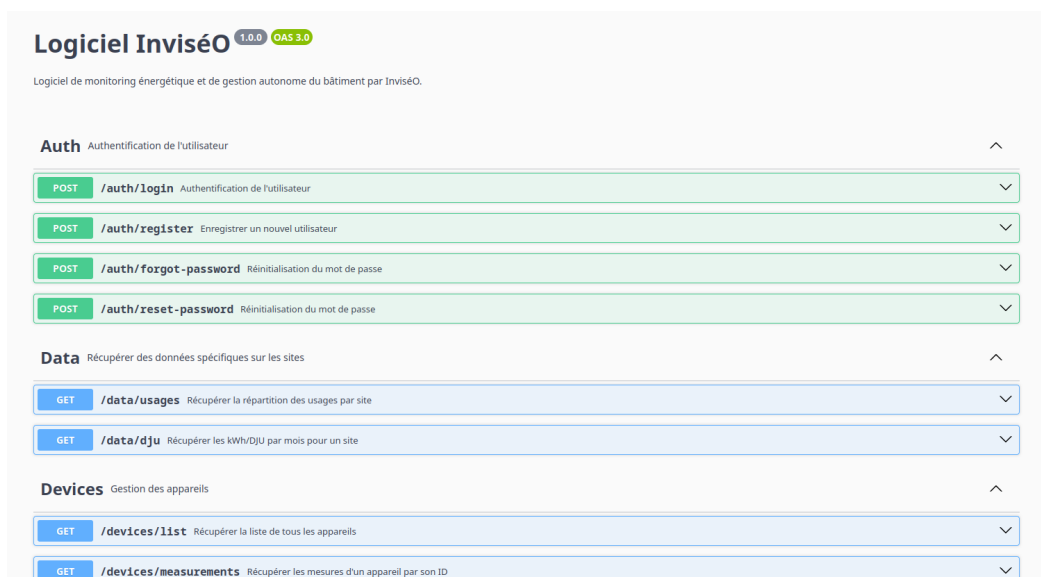


FIGURE 10 – Page de Swagger affichant la liste des routes de l'API

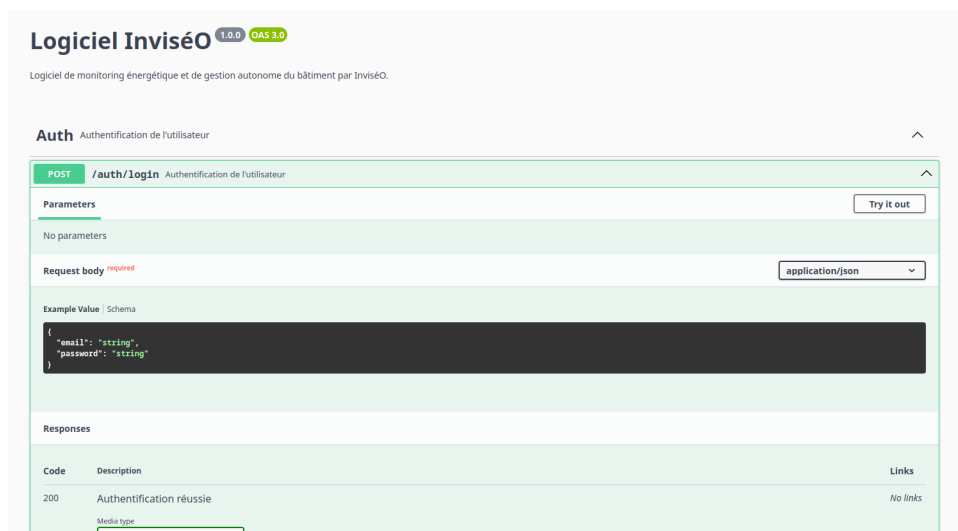


FIGURE 11 – Page de Swagger affichant les paramètres d’entrée et résultats de sortie de la route /api/auth/login

3 Bilans

3.1 Bilan général de l’alternance

Globalement, je pense avoir été à la hauteur du niveau de compétences techniques demandées par Inviséo. Je trouve que les travaux accomplis depuis le début de la seconde période se sont révélés très largement au dessus de ce qui serait estimé pour une équipe de deux personnes, démarrant un projet « de zéro » (nous avons démarré avec les connaissances acquises lors de nos mois d’expérience à l’UPPA.). Actuellement, le logiciel, à l’aspect tout de même légèrement moins industriel et conformiste, commence à concurrencer dans les grandes lignes ce qui peut être proposé par certaines des plus grosses ESN (Entreprise de Services du Numérique) voire même les géants déjà installés et réalisant des projets de confiance, déjà éprouvés et largement distribués.

La charge de travail a été certes intense, mais extrêmement formatrice. Nous avons dû nous adapter aux contraintes de temps, à la demande de nos clients, et dans des domaines extrêmement variés, avec des acteurs parfois complètement étrangers au domaine de la consommation énergétique et du bâtiment (des élus principalement), des acteurs spécialistes dans certains domaines bien précis qui sont touchés par notre application, comme la GTB (Gestion Technique du Bâtiment) et la GTC (Gestion Technique Centralisée), mais aussi savoir convaincre les experts responsables de la surveillance de la consommation énergétique des bâtiments de nos clients d’utiliser une solution comme la nôtre.

Enfin, je ne souhaite pas cacher ma fierté quant à ma contribution à une telle entreprise. C’est pour moi un réel accomplissement, d’autant plus que les défis se sont révélés nombreux, et ont tous connu un franc succès. Ce projet représente pour moi bien plus qu’un simple logiciel de monitoring, sinon une brique supplémentaire à l’édifice global du combat contre le changement climatique. L’informatique, bien qu’elle puisse parfois contribuer au problème, a un rôle clé à jouer dans les actions environnementales nécessaires pour les années, décennies voire siècles à venir.

3.2 Compétences du B.U.T.

- **C1 - Réaliser un développement d'application** : J'ai analysé, conçu et programmé certaines parties de la plateforme SaaS Inviséo dédiée au monitoring énergétique et à la gestion technique du bâtiment. Cela a inclus la conception et le déploiement d'une stack MEAN (MongoDB, Express, Angular et NodeJS). Un accent particulier a été mis sur l'API Inviséo (le backend) ainsi que sur les relais esclaves, permettant de renvoyer des données spécifiques des bâtiments via Modbus et Webservice pour des données de confort ou de sous-comptage.
- **C2 - Optimiser des applications** : J'ai dû gérer de manière avancée les erreurs et développer des programmes légers pour les Inviséo Box, en minimisant les informations demandées et envoyées, pour éviter la surcharge vers le serveur central. J'ai également utilisé les API InfoClimat et Enedis, et intégré des fonctionnalités comme l'enregistrement de leurs données afin d'éviter les réponses trop volumineuses, et souvent trop lentes. La séparation des responsabilités a été effectuée grâce à la conteneurisation, rendant nos programmes plus légers et extrêmement portables.
- **C6 - Collaborer au sein d'une équipe informatique** : Je pense avoir été en capacité de comprendre les besoins cruciaux de nos clients et proposer un réajustement pertinent de la perspective de développement dans le cadre de nos contrats. Nous avons été confrontés à une difficulté contractuelle due à une erreur d'évaluation des capacités du logiciel précédemment utilisé, ce qui a nécessité une prise de décision urgente par rapport aux *deadlines* imposées. La collaboration entre Vincent et moi, en tant que développeurs, a été constante grâce à notre dépôt GitHub et à un tableau Kanban en ligne, permettant un suivi systématique de nos tâches. En plus de cette communication inter-développeurs, nous avons aussi collaboré avec Pierre et Guillaume, qui ont un point de vue beaucoup moins « informatique », et détiennent des connaissances beaucoup plus poussées concernant la GTB, ainsi que tout ce qui touche de près ou de loin à l'énergétique du point de vue du bâtiment. Ces compétences complémentaires ont été, sont, et seront toujours cruciales à la bonne réussite du projet.

3.3 Perspectives professionnelles

Comme énoncé précédemment, l'expérience de mon alternance chez Invisio fut extrêmement enrichissante et formatrice.

Cependant, malgré la satisfaction et l'enthousiasme que j'ai ressentis en travaillant sur le développement de la solution Invisio, cette expérience m'a également permis de clarifier mes aspirations professionnelles. En effet, je ne souhaite pas poursuivre dans le domaine du développement logiciel à proprement parler. Depuis mes années de lycée, mon attrait pour l'informatique s'est toujours porté sur les concepts plus abstraits, les principes fondamentaux et les théories qui structurent ce domaine. Ce sont ces aspects qui définissent l'informatique comme « la science du traitement automatique de l'information », correspondant davantage à la science informatique qu'à l'ingénierie informatique (distinction bien plus marquée chez nos homologues anglophones que dans la langue de Molière). Mon véritable désir est de gagner en abstraction, d'explorer les fondements théoriques de l'informatique plutôt que de me concentrer sur les aspects programmatoires.

J'avais déjà fait part de cette volonté d'orientation vers une compréhension plus conceptuelle et théorique de l'informatique, l'année dernière, dans le bilan de mon rapport de stage, alors effectué dans le laboratoire de recherche en informatique de Domolandes. Cette idée s'est encore renforcée cette année.

Bien que mon alternance chez Invisio ne m'ait pas rapproché directement de cet objectif professionnel axé sur la recherche, elle m'a néanmoins offert une précieuse expérience sur le terrain. J'ai gagné en agilité et en polyvalence, développant des compétences pratiques en informatique qui seront indéniablement utiles pour mes futurs travaux. Cette agilité acquise, combinée à une meilleure compréhension des défis techniques et des solutions pratiques, enrichira ma démarche de recherche et facilitera la réalisation de mes ambitions professionnelles.

En conclusion, cette année d'alternance a été un tremplin important dans mon parcours. Elle m'a permis de consolider mes compétences techniques tout en clarifiant mes aspirations académiques et professionnelles, orientées vers la recherche et l'exploration théorique des concepts informatiques. Je suis déterminé à poursuivre cette voie, convaincu que cette combinaison d'expérience pratique et d'intérêt théorique sera la clé de mon épanouissement professionnel futur.

3.4 Bilan personnel sur mon parcours en B.U.T.

Finalement, je tenais aussi tout particulièrement à réaliser un bilan quant à mon parcours en B.U.T., puisque je pense qu'il est important de prendre du recul et de réaliser un bilan global sur mon expérience de formation. J'estime que l'intégrer dans le rapport dédié à l'alternance, réalisée lors de la dernière année est la meilleure manière de conclure (ou presque) ces trois années de formation.

Comme je l'avais également précisé dans mon rapport de stage de l'année dernière, ma voie dans l'informatique n'a jamais été tracée depuis le départ. Je n'ai jamais eu une aisance naturelle avec tout l'aspect très pragmatique de l'utilisation efficace d'un ordinateur (comme en témoigne ma première expérience avec Linux, en terminale, via un émulateur de terminal en ligne, où j'avais été incapable de suivre le premier cours d'introduction aux systèmes Linux, simplement car les caractères ne s'affichaient pas lors de la saisie du mot de passe...), et n'ai jamais d'ailleurs véritablement souhaité être « développeur » comme peut-être une grande partie voire majorité d'étudiants en B.U.T. Informatique.

Ce qui m'attirait réellement était ces concepts (abordés dans la section 3.3 de ce rapport) plus abstraits. Pour cela, il aurait très probablement été préférable de suivre un parcours plus académique, dont la première porte d'entrée est la généralement une licence. Bien qu'un B.U.T. peut-être une formation envisagée pour ce genre de parcours, et notamment avec deux années au contenu dense

et intéressant, je considère le choix de poursuivre en troisième année de B.U.T comme un échec personnel, qui n'aura pas été en adéquation avec mes ambitions professionnelles. Cet échec, dû à un manque de motivation et à la peur de changer de formation « en cours de route », aurait pu être évité pour un choix bien plus cohérent par rapport à mes ambitions.

À l'heure actuelle, les possibilités quant à ma poursuite d'études restent incertaines. Cependant, je suis dans l'attente de différentes propositions d'admissions pour poursuivre mes études dans un parcours plus académique, en accord avec mes ambitions professionnelles. Une possibilité serait de redoubler une année pour rejoindre une troisième année de licence, ce qui me donnerait l'opportunité de renforcer mes compétences en informatique fondamentale et de me préparer à un master dans ce domaine. Une autre possibilité serait de rejoindre directement un master en "Informatique fondamentale et appliquée", ce qui me permettrait d'approfondir mes connaissances dans les domaines qui m'intéressent le plus. Ces deux options seraient pour moi une source de grande satisfaction et me permettraient d'ajuster le tir vers un parcours plus académique, en adéquation avec mes aspirations professionnelles. Quelle que soit la voie qui s'offrira à moi, je suis déterminé à faire tout mon possible pour réussir et atteindre mes objectifs professionnels.

Finalement, je tiens à souligner que je ne regrette en aucun cas mon parcours en B.U.T., puisqu'il m'a apporté des compétences indéniables en informatique, et m'a permis d'avoir, comme susmentionné, une agilité très intéressante concernant énormément de concepts applicatifs de l'informatique, qui me permettront plus tard de détenir plus d'abstraction en général, et également lors de mes futurs travaux.

Le B.U.T. m'a permis de gagner en maturité, sur le plan social, en me faisant rencontrer des personnes très intéressantes, des amis, de beaucoup d'horizons et avec des points de vue très intéressants. De plus, je souhaite remercier avec cœur l'ensemble de l'équipe pédagogique et particulièrement professeurs, pour avoir agréablement dispensé des cours de qualité, toujours en réalisant des efforts importants à l'égard des étudiants.

Références

- Dunka, B., Emmanuel, E., and Oyerinde, Y. (2018). Simplifying web application development using-mean stack technologies. 04.
- Lavayssiere, C. (2022). *Supervision et instrumentation à distance, libre et interopérable, du capteur à l'utilisateur*. Theses, Université de Pau et des Pays de l'Adour.